

ForkServe

分支进 kernel 之前，决定这条 residual 要不要 prefill、要不要过 connector

一次 fan-out: trunk 长度 L , k 条 residual ℓ_i , 最后一条 committed spine

背景

Tree of Thoughts、ESC /
Speculative Rejection / DPTS、
APC 与 DistServe / Splitwise 各
自在哪一步付钱。

设计

CoW context tree、五动作
admission、probe、connector 只
搬 Keep。

结果

2026-10-01 prune ablation,
2026-09-29 三轮 ToT。
Qwen3-8B / 4B, A100-40GB。

一次 fan-out 的三笔账

Agent 和 Tree-of-Thoughts 的服务单位是一次分支：共享一条 trunk，分叉出 k 条短 residual，最后只提交一条 spine。传统栈对这次分支收三笔钱。

Work, GPU prefill

第一次 expand 时 block 还没进 radix index。APC 和 hash prefill 把 trunk 克隆 k 份。

$$W = kL + \sum_i \ell_i$$

Xfer, connector

DistServe / Splitwise 把 prefiller 算出的每一页传给 decoder。Aborted residual 也在这条链路上。

Decode-to-cut

ESC、Speculative Rejection、DPTS 的分数来自已经 resident 的 token。Prefill 和一段 decode 发生在丢弃之前。

后面的设计对着这三笔账

Alias 去掉 W 和 X 里的 L 。Admission 把不该留的 residual 挡在 kernel 和 connector 之前。Abort 之后活着的 KV 只剩 spine。

Tree of Thoughts 具体怎么搜

Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023.

步骤	具体做法	对 serving 的含义
Thought	中间状态是一段推理。从当前 state 采样 k 个 candidate thought。	k 条 residual 共享父 prompt，分叉只在短后缀。
Value	对已经生成完的 thought 打分：value prompt，或多次投票。	分数读的是已经 decode 出来的文本。
Search	BFS 保留 top- b beam；DFS 先走最高分，低于阈值就剪。	剪枝发生在这一步生成结束之后。
下一层	被留下的 beam 各自成为下一层的父节点，再展开。	每一层都要把 k 份上下文重新交给引擎。

ToT 把搜索单位从 token 抬到 thought。引擎侧仍是 k 次独立 generate。Beam 被丢掉时，这条 thought 的 prefill 和这一步的 decode 已经付过。同方向：Hive (Luo et al., arXiv 2026) 在算法层和任务层扩 branching，KV 仍按请求隔离。

三种 decode-time pruning，各自停在哪

三条线都在模型已经写出一段 token 之后才决定丢掉谁。差别是这段前缀有多长。

方法	出处	具体规则	丢掉之前必须生成
ToT beam	Yao et al., NeurIPS 2023	一步 thought 生成完再打分，留 top beam。	这一步的完整 decode
ESC	Li et al., ICLR 2024	Self-consistency 抽多个完整样本。窗口 w 内答案一致，才跳过后面的样本。	窗口里每个样本的全程 decode
Speculative Rejection	Sun et al., NeurIPS 2024	Best-of- N 。生成到 partial-reward horizon τ ，按部分奖励排序，停掉较低分位 α 。	每个候选的 prefill + τ 个 token
DPTS	Ding et al., ACL 2025	每个活跃节点先生成固定 mini-step，再按置信度 early-stop 低分孩子。	每个节点的 prefill + mini-step

在我们的 fan-out 上，窗口经常关不上

ESC 的窗口是 5 个一致样本。 $k=4$ 且有两条低分时，窗口一直开着，四个孩子都 decode 到预算。Speculative Rejection 的 memory-limit 触发器在 $k=4$ 不会响，更早的检查点是 $\tau=256$ 。DPTS 最早，mini-step 仍是 100 token。

Prefix cache 与 P/D 分离具体做什么

赢家之后再开一次请求，APC 可以 hash-hit。第一次 expand 本身省不下来。

三条线停在同一个地方

共享 trunk 的系统等块写完才发现共享。丢弃分支的系统等 token 写完才打分。P/D 把这两件事的产物整页搬走。

方法	第一次 expand	最早能丢掉非赢家	Connector
Recompute	k 份 L	请求结束	无
APC / radix / hash prefill	k 份 L	块发布之后的 LRU	无；赢家重放才是 hit
P/D disaggregation	k 份 L	insert 之后	$kL + \sum_i \ell_i$
ESC / SR / DPTS	每条 residual	一段已生成前缀之后	每个已 prefill 孩子的 KV
ForkServe	L 的一份 alias	loop 在 kernel 前；其余低分在 t_0 probe 后	Keep 的 miss tail

占用也是三档

$M_{\text{clone}} = b(kL + \sum_i \ell_i)$, $M_{\text{CoW}} = b(L + \sum_i \ell_i)$, $M_{\text{spine}} = b(L + \ell_\star)$ 。APC、recompute、baseline disaggregation 的第一次 expand 落在 clone。Abort 之后该留下的是 spine。

四条核心设计

ForkServe 坐在 agent runtime 和 LLM engine 之间。Harness 交出分支结构；控制面负责 admit、prune、bind、retain；backend 搬 page 和 token。用户可见 token 的关键路径上只有 commit。

1 CoW context tree

fork 只写两个字：parent 指针和 alias_len，并对被 alias 的 frame 做 incref。 $O(1)$ 于 L 。Abort 只还 residual 的 page。

3 Probe，而不是一刀切

重复 loop 在 prefill 前丢掉。其他低分先 decode t_0 个 token，probe 分数过 τ 才继续到预算 D 。Index 0 永远留在 Keep。

2 五动作 admission

先读 radix 命中 m_i ，再读是不是重复 loop。一次 first-match 同时定下 W_i 和 X_i 。Trunk 的 L 两边都不出现。

4 Connector 与调度看同一个 Keep

X 只加 Keep 里真正 miss 的尾巴。Abort 后池子按 spine 计并发。Chunk 预算先给 committed decode，剩下的才去 prefill 下一次 miss。

Copy-on-write context tree

一个 session 是一棵有根树。节点保存 token 序列、相对父节点的 residual、模式 (Spec / Commit / Idle / Dead)，以及逻辑页表：parent、alias_len、自己的 page。

不变量

- 父序列是子序列的前缀。
- 除 join 期间，一个节点最多一个 committed child。
- Speculative 节点对 sampler 不可见。
- 一页的 refcount 等于还映射着它的非 dead 节点数。

多层 fan-out 是一串 CoW。活路径共享公共前缀上的每一页。没有活孩子的子树级联 abort，停在仍有活孩子的节点。

API。commit 才在关键路径上

调用	作用
open(session)	建根, prefill system + user
fork(u, bid, tok?)	u 的孩子; 可选已知后缀
speculate(v, pri)	标记 v 可以投机 prefill
commit(u, tokens)	按 LCP 绑定, 开始 decode
join(children, pol)	all / first / k-of-n
abort(v) / close	丢掉子树 / 结束 session

Abort 还是 residual

释放 $b \cdot \ell_i$ ，而不是 $b \cdot (L + \ell_i)$ 。LCP commit 把赢家的整页发布进 APC index，后面的 session 还能 hash-hit。Spec page 自己不进 index。

一个 first-match 动作，同时决定 W 和 X

m_i 是 residual 里已经在 radix index H 中、按页对齐的最长前缀。读 m_i 不花模型 FLOPs。只有 miss 才看这条是不是重复 loop。

动作	条件	Work	Xfer
HS, hash skip	$m_i = \ell_i$, 整段命中	0	0
HP, hash partial	$0 < m_i < \ell_i$	$\ell_i - m_i$	留在 Keep 才搬 miss
DS, draft skip	$m_i = 0$ 且 $i \in \Lambda$ (重复 loop)	0	0
EA, early abort	miss、不是 loop、前 α 段分数 $< \tau$	$\lceil \alpha \ell_i \rceil$	0
PF, prefill	miss、不是 loop、分数过线	ℓ_i	留在 Keep 才搬

Index 0 强制进 Keep, 所以 $a_0 \in \{\text{HS}, \text{HP}, \text{PF}\}$ 。Alias 已经把 trunk 指到父页表, 所以

$L \notin W_i$, 也 $L \notin X_i$ 。Baseline disaggregation 搬的是 $kL + \sum_i \ell_i$ 。冷 index 上

$W_{\text{APP}} \leq W_{\text{CoW}} \leq W_{\text{APC}} = W_{\text{PD}}$ 。有一条 loop 被 draft-skip, 则 W 严格更小; Keep $\neq [k]$ 或有一条命中, 则 X 严格小于 PD。

Loop 直接丢，其余低分只探 t_0

把所有 $s_i < \tau$ 都挡在 prefill 前，会把还能做对的 illegal text 一起丢掉。Probe admission 收成三条。

分支	Prefill 分数	动作	Decode 长度
$i \in \Lambda$, 重复 loop	约 0.02	DS, kernel 之前丢掉	0
$i = 0$, 或 $s_i \geq \tau$	策略分支约 1.0	PF	预算 D
其他低分	illegal text 约 0.22	先 PF, 再 decode t_0	$s'_i \geq \tau$ 则续到 D , 否则停在 t_0

两个阈值不是一回事

Prefill 阈值默认 $\tau_{\text{pre}} = 0.15$, 只掉 loop。Decode probe 在 MATH-500 上是 $\tau = 0.45$, $t_0 \in \{64, 128\}$ 。Host 的 τ 、DPTS step、 α 都不参与 prefill admission。

剩余算力留给下一次 miss

Abort 之后 $M = b(L + \ell_*)$, 并发槽位是 $\lfloor H/M \rfloor$ 。Chunk 预算先满足 committed decode, 剩下的 B^s 去 prefill 下一条 miss, 或在 idle 够长时 prefill 已知后缀。投机 token 不进 sampler。

Prefill 剪枝比 decode 剪枝少算了什么

Qwen3-8B, GSM8K, $n=16$, $D=512$, $tp=1$, 四张 A100-40GB。 $k=16$ 时三个 host 单独跑, computed prefill 都是 5008。 Decode 阶段的剪枝没有少掉任何一次 residual prefill。

	Host 单独	$+\tau_{pre}=0.15$	变化
Admitted residual	16	12	-4
Computed prefill	5008	3848	-23%
ESC decode tokens	131072	98304	-25%
ESC 端到端	17.05 s	14.13 s	-17%

代价模型把同一笔账算得更极端: ESC 要 decode

20480 token 才关窗 (276 ms), DPTS 的 100-token mini-step 仍在被丢掉的孩子上花 2000 token (78.7 ms)。 ForkServe+ 在那些 residual 上的 decode 是 0, fan-out 31.4 ms。 GPU 上, 被 admission 丢掉的孩子不再占 prefill, 也不再占 decode slot。 来源: logs/prune_ablation_thresh/all.json, 2026-10-01。 端到端 = trunk warm + fan-out。 三条 host 的 prefill token 相同。

同一个阈值插在三个 host 前面

τ_{pre} 不读 host 的参数，所以三个 host 的 admitted 和 computed prefill 完全一样。端到端的差，来自这个 host 本来会把 loop decode 多久。

$k=16$, 秒	单独	+0.15	+0.45	any-finished
ESC	17.05	14.13	10.92	全是 16/16
Speculative Rejection	13.14	12.47	10.80	全是 16/16
DPTS	11.86	11.49	10.90	全是 16/16

k	ESC	ESC+0.15	ESC+0.45	SR	SR+0.15	DPTS	DPTS+0.15
4	16/16	15/16	14/16	13/16	13/16	13/16	13/16
8	16/16	16/16	14/16	15/16	16/16	15/16	16/16
16	16/16	16/16	16/16	16/16	16/16	16/16	16/16

$k=8$ 时 SR 和 DPTS 单独是 15/16;

前面加上 0.15 变成 16/16。丢掉的是 loop。 $k=16$ 上 0.45 仍是 16/16, ESC 从 14.13s 降到 10.92s。 $k=4$ 和 $k=8$ 上 0.45 会再掉 illegal text。默认 0.15, 因为每个 k 都守住精度。 来源:

logs/prune_ablation_thresh/all.json。 $k=4/8/16$ 的 prefill token: 1320/2624/5008 $\rightarrow$ 1018/2020/3848 (0.15)
 $\rightarrow$ 732/1448/2768 (0.45)。

为什么不能把低分全部挡在 prefill 前

MATH-500 level ≥ 4 , Qwen3-8B, $n=32$, $D=1024$, $\tau=0.45$ 。一半 residual 是竞赛策略，另一半在重复 loop 和 illegal text 之间交替。Any-finished: 某条跑满预算的分支对上参考答案。

策略	$k=4$	$k=8$	$k=16$
Full fan-out	12/32, 25.3 s	15/32, 52.5 s	20/32, 104.1 s
丢掉全部 $s < \tau$	6/32, 19.1 s	11/32, 25.6 s	13/32, 52.1 s
只跳过 loop	11/32, 23.0 s	17/32, 41.5 s	16/32, 71.4 s
Grow $t_0=64$	8/32, 24.0 s	16/32, 42.0 s	18/32, 72.5 s
Grow $t_0=128$	9/32, 24.7 s	13/32, 43.3 s	20/32, 72.2 s
Loop 也拿去 probe	10/32, 24.9 s	15/32, 43.3 s	20/32, 74.4 s

一刀切省时间，也剪掉正确答案

$k=16$ 把所有 $s < \tau$ 挡在 prefill 前，any-finished 从 20/32 掉到 13/32。Loop 在每个 k 上都是 0 分，所以 DS 是安全的。 $t_0=128$ 把 illegal residual 全部延长，loop 一条都不延长，精度回到 20/32，时间从 104.1s 降到 72.2s，decode token 从 524288 降到 393216。把 loop 也拿去 probe，一条都续不上，同样 20/32 要 74.4s。

Grow 行: logs/prune_ablation_grow/shard*.json (2026-10-01), $k=16$ grow@128 为 72.21s、20/32。Full / 一刀切 / 只跳过 loop 来自论文 tab:grow, 同一次实验。

P/D 分离之后, pruning 少搬了什么

DistServe 和 Splitwise 的 connector 搬运 prefiller 已经算出的页。分支上的 pruning 如果发生在 insert 之后, 被丢掉的 residual 已经过了网。Admission 把 Keep 定在 insert 之前,

	APC	P/D	FS CoW	ForkServe+	
connector 只看见留下的 miss。	Prefill tokens W	11520	11520	3840	2592 Alias
	Connector tokens X	0	11520	0	32
	Fan-out	139 ms	162 ms	47 ms	31.4 ms

去掉的是 k 份 trunk: W 从 11520 降到 3840, 四条 residual 仍全部 prefill。Admission 再把 30/40 条低分 residual 挡在 kernel 外, connector 从 11520 token 降到 32。相对 APC, prefill token -77% 。GPU 上的对应物是三轮 ToT: ForkServe+ 的 transfer_tokens 是赢家的 miss (Qwen3-8B GSM8K 为 1402), 192 条 residual 里 144 条没有 prefill。控制面代价模型, 不是 GPU wall clock。10 sessions, $k=4$, $L=256$, $\ell=32$ 。Prefill $12\ \mu\text{s}/\text{token}$, 传输 $2\ \mu\text{s}/\text{token}$ 。论文 tab:app。

三轮 ToT: expand 为什么变成 8-10×

一个 session、 $k=4$ 个 agent、三轮，每个问题自己一次 generate。前两轮只 decode 32-token 的 step，输家 abort，赢家 spine 是下一轮的父节点。第三轮才是答案。fork 是 $O(1)$ alias,

expand 不再复制 trunk。

Expand, ms / 题	4B GSM8K	4B Game24	8B GSM8K	8B Game24
APC	429	435	597	598
ForkServe	83	87	94	94
ForkServe+	55	53	58	56
ForkServe+ / APC	7.9×	8.2×	10.2×	10.6×

192 → 48

三轮发出的 residual 里，
ForkServe+ 真正 prefill 的条数。

Peak 是 spine

GSM8K 275/318 (0.86×)。
Game24 216/265 (0.82×)。两
个模型一样。

端到端看 decode token

ForkServe 贴着 APC
(1.00–1.06×)。ForkServe+ 在发
出的 token 变少时才拉开。

来源: logs/tot_mt/qwen3_{4b,8b}/eval.json, 2026-09-29. $n=16$, $tp=2$, one problem per generate. 毫秒 = fanout_ms/16. 8B GSM8K 端到端 2.48 → 1.91s (decode token 3594 → 2485); 4B GSM8K token 几乎不变 (3421 → 3334), 端到端 1.66 → 1.63s。

精度有没有被剪掉

三轮 ToT 的质量是 exact match (Game24 是 success rate)。Prefill 剪枝的质量是 any-finished。两套数字回答的是不同的问题。

三轮 ToT, $n=16$	APC	ForkServe	ForkServe+	Peak KV
Qwen3-4B GSM8K	15/16	15/16	15/16	318 $\rightarrow$ 275
Qwen3-8B GSM8K	16/16	15/16	15/16	318 $\rightarrow$ 275
Qwen3-4B Game24	6/16	8/16	9/16	265 $\rightarrow$ 216
Qwen3-8B Game24	9/16	9/16	10/16	265 $\rightarrow$ 216

$\tau_{\text{pre}}=0.15$ 守住 GSM8K

$k=8$ 和 $k=16$ 都是 16/16, $k=4$ 是 15/16。 $k=8$ 上它把 SR 和 DPTS 从 15/16 抬回 16/16。 $k=16$ 上 $\tau=0.45$ 同样 16/16, 时间更短。

只留 1 条会伤精度

prune_ablation_wide 里没有 score admit mode 的 app, 在 $k=16$ 、 $\tau=0.45$ 时只 admit 1 条, prefill 降到 414。ESC 的 survivor 掉到 11/16, SR 掉到 9/16。同一阈值下 score 模式 admit 8 条, survivor 保持 16/16。

ToT: $\text{logs}/\text{tot_mt}, \text{task_score} \times 16$ 。AIME / MATH / SVAMP / AMC 的 ForkServe 行用了 slot-refill, 和 APC 的 one-per-generate 不是同一调度, 不放在一起比。

1. ToT 的 beam、ESC 的一致窗口、Speculative Rejection 的 τ 、DPTS 的 mini-step，分数都来自已经 resident 的 token。DistServe / Splitwise 接着把这些页全部传走。APC 要等块写完才发现 trunk 是共享的。
2. ForkServe 用一次 alias 去掉 L ，用一次 first-match 决定这条 residual 是命中、部分命中、loop、提前停、还是 prefill。Loop 在 kernel 前丢掉；其他低分只给 t_0 的 probe。Connector 只搬 Keep 的 miss。
3. GPU 上， $\tau_{\text{pre}}=0.15$ 把 $k=16$ 的 computed prefill 从 5008 降到 3848，any-finished 保持 16/16，ESC 从 17.05s 降到 14.13s。三轮 ToT 里 192 条 residual 只 prefill 48 条，expand 是 APC 的 7.9–10.6 倍，peak KV 是 spine。

数字来自 logs/prune_ablation_{thresh,grow,wide} (2026-10-01) 和 logs/tot_mt (2026-09-29)。P/D 的 139 / 162 / 47 / 31.4 ms 是控制面代价模型。MATH-500 的 full fan-out 与“丢掉全部 $s < \tau$ ”来自论文 tab:grow, grow 行已用 shard JSON 核对。